

Venus Labs - PrimeV2 and Prime Leaderboard contracts

Security Assessment

CertiK Assessed on Jun 10th, 2026





Certik Assessed on Jun 10th, 2026

Venus Labs - PrimeV2 and Prime Leaderboard contracts

The security assessment was prepared by Certik.

Executive Summary

TYPES

Lending

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 06/03/2026

Final report published on 06/10/2026

Vulnerability Summary



11

Total Findings

5

Resolved

0

Partially Resolved

6

Acknowledged

0

Declined

1 Centralization

1 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

2 Medium

1 Resolved, 1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

4 Minor

2 Resolved, 2 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

4 Informational

2 Resolved, 2 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Overview

[PR-670](#)

Dependencies

[Third Party Dependencies](#)

[Protocol Dependencies](#)

[Recommendations](#)

Findings

[VLP-01 : Centralization Related Risks](#)

[VLP-02 : Residual Reward Claims Can Clear Unindexed PLP Rewards](#)

[VLP-03 : Partial Score Updates Can Misallocate Prime Rewards](#)

[VLP-04 : Tier-Based Deposit Compaction Can Understate Remaining Effective Stake After Partial Withdrawals](#)

[VLP-05 : Precision Truncation In `accrueInterest` Can Permanently Drop Small Reward Slices](#)

[VLP-06 : Non-BSC PrimeV2 Deployments Can Fail Due To Required `Unitroller` Dependency Resolving To Zero Address](#)

[VLP-07 : PrimeV2 Deploy Script May Attempt ACM-Gated Wiring Before VIP Permissions Are Granted](#)

[VLP-08 : Zero Oracle Price Allows Uncapped PrimeV2 Score Capital](#)

[VLP-09 : Required Timelock Address Resolution Fails Open To Zero Address](#)

[VLP-10 : PrimeV2 Deploy Script Uses A Local Deployment Dependency In Live Deployments](#)

[VLP-11 : PrimeLeaderboard Can Be Initialized With Stale Or Inconsistent State During VIP Migration](#)

Appendix

Disclaimer

CODEBASE | VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

Repository

Base: <https://github.com/VenusProtocol/venus-protocol/pull/670>

UpdateV1: <https://github.com/VenusProtocol/venus-protocol/tree/feat/VPD-1366>

Commit

Base: [e421dda8598abe8d1447f0bedf9ae019c4c83780](https://github.com/VenusProtocol/venus-protocol/commit/e421dda8598abe8d1447f0bedf9ae019c4c83780)

UpdateV1: [a1fc82948cc768b8e9a980b553f8b3aef6dd77a7](https://github.com/VenusProtocol/venus-protocol/commit/a1fc82948cc768b8e9a980b553f8b3aef6dd77a7)

AUDIT SCOPE

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

CertiKProject/certik-audit-projects

-  contracts/Tokens/Prime/PrimeV2.sol
-  contracts/Tokens/Prime/PrimeLeaderboard.sol
-  deploy/014-deploy-prime-v2.ts
-  deploy/015-configure-prime-v2.ts
-  contracts/Tokens/Prime/PrimeV2Storage.sol
-  contracts/Tokens/Prime/IPrimeLeaderboard.sol
-  contracts/Tokens/Prime/PrimeLeaderboardStorage.sol
-  contracts/Tokens/Prime/IPrimeV5.sol
-  contracts/Tokens/Prime/Interfaces/IPrimeV2.sol

APPROACH & METHODS

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

This audit was conducted for Venus Labs to evaluate the security and correctness of the smart contracts associated with the Venus Labs - PrimeV2 and Prime Leaderboard contracts project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

OVERVIEW

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

This audit concerns the changes made in the in scope files outlined in the following PR:

- [PR-670](#)

Note that any centralization risks present in the existing codebase before this PR were not considered in this audit and only those added in this PR are addressed in the audit. We recommend all users carefully review the centralization risks, much of which can be found in our previous audits, which can be found here: <https://skynet.certik.com/projects/venus>.

PR-670

PR-670 introduces PrimeV2 and PrimeLeaderboard, updating the Venus Prime reward system with leaderboard-based XVS staking eligibility, time-weighted effective stake calculation, permissionless Prime token minting, batched score updates, and migration support for existing XSVVault stakers. The PR also adds deployment and configuration scripts for deploying the new Prime contracts and preparing them to be wired into the existing PrimeLiquidityProvider, XSVVault, and core pool Comptroller integrations.

DEPENDENCIES

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

Third Party Dependencies

The protocol is serving as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- ERC20 Tokens

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Protocol Dependencies

PR-670 depends on correct integration with existing Venus protocol components, including `PrimeLiquidityProvider`, `XVSVault`, the core pool `Comptroller`, `ResilientOracle`, and `AccessControlManager`. These contracts are assumed to be correctly deployed, configured, permissioned, and available when PrimeV2-related actions are performed.

Because PrimeV2 spans multiple existing protocol components, deployment ordering, VIP execution, migration steps, and future configuration updates should be coordinated carefully. Incorrect sequencing, missing permissions, stale deployment metadata, or inconsistent configuration across these components may affect PrimeV2 reward accounting, eligibility checks, hook routing, or operational readiness.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state.

FINDINGS

VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS



11

Total Findings

0

Critical

1

Centralization

0

Major

2

Medium

4

Minor

4

Informational

This report has been prepared for Venus Labs to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 11 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VLP-01	Centralization Related Risks	Centralization	Centralization	● Acknowledged
VLP-02	Residual Reward Claims Can Clear Unindexed PLP Rewards	Logical Issue	Medium	● Resolved
VLP-03	Partial Score Updates Can Misallocate Prime Rewards	Design Issue	Medium	● Acknowledged
VLP-04	Tier-Based Deposit Compaction Can Understate Remaining Effective Stake After Partial Withdrawals	Design Issue	Minor	● Acknowledged
VLP-05	Precision Truncation In <code>accrueInterest</code> Can Permanently Drop Small Reward Slices	Coding Issue	Minor	● Resolved
VLP-06	Non-BSC PrimeV2 Deployments Can Fail Due To Required <code>Unitroller</code> Dependency Resolving To Zero Address	Inconsistency	Minor	● Acknowledged
VLP-07	PrimeV2 Deploy Script May Attempt ACM-Gated Wiring Before VIP Permissions Are Granted	Inconsistency	Minor	● Resolved
VLP-08	Zero Oracle Price Allows Uncapped PrimeV2 Score Capital	Design Issue	Informational	● Acknowledged
VLP-09	Required Timelock Address Resolution Fails Open To Zero Address	Inconsistency	Informational	● Acknowledged

ID	Title	Category	Severity	Status
VLP-10	PrimeV2 Deploy Script Uses A Local Deployment Dependency In Live Deployments	Logical Issue	Informational	● Resolved
VLP-11	PrimeLeaderboard Can Be Initialized With Stale Or Inconsistent State During VIP Migration	Logical Issue	Informational	● Resolved

VLP-01 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization		● Acknowledged

Description

This section summarises the centralization surface of the in-scope Prime V2 contracts. Because the same surface may overlap with earlier Venus reviews, we recommend reading this list together with the prior CertiK audits at <https://skynet.certik.com/projects/venus> to ensure no item is missed.

Prime V2

PrimeV2

In the contract `PrimeV2`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call administrative functions including:

- `issue(address, bool)`
- `issueBatch(address[], bool[])`
- `burn(address)`
- `burnBatch(address[])`
- `addMarket(address, uint256, uint256)`
- `removeMarket(address)`
- `updateAlpha(uint128, uint128)`
- `updateMultipliers(address, uint256, uint256)`
- `setPrimeLiquidityProvider(address)`
- `setComptroller(address)`
- `setOracle(address)`
- `setPrimeLeaderboard(address)`
- `setMaxLoopsLimit(uint256)`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted these privileges may allow the attacker to take advantage of this authority and do the following:

- Issue Prime tokens directly to arbitrary users, including bypassing the normal `claimPrime()` leaderboard eligibility path.
- Burn Prime tokens from arbitrary users, removing their Prime status and disabling their future Prime reward participation until restored.

- Add new Prime markets or remove existing Prime markets, changing which vToken markets participate in Prime rewards.
- Configure malicious or incorrect market multipliers, causing reward scores to be inflated, suppressed, or redistributed unfairly across Prime users.
- Change `alpha`, altering the score formula used to calculate Prime reward shares.
- Replace the `primeLiquidityProvider`, redirecting or disrupting the source of reward funding used by Prime markets.
- Replace the `Comptroller`, allowing PrimeV2 to trust an attacker-controlled or incorrect market-support source.
- Replace the oracle, allowing PrimeV2 to calculate user scores from manipulated or invalid prices.
- Replace the `PrimeLeaderboard` reference, changing the contract used for `claimPrime()` eligibility checks.
- Set the loop limit to values that can disable large batch operations or allow excessively expensive batch calls.

In the contract `PrimeV2`, the `owner` account also has authority inherited through the upgrade/access-control base contracts, including ownership transfer and access-control-manager configuration paths.

Any compromise to the `owner` account may allow the attacker to take advantage of this authority and do the following:

- Transfer ownership to an attacker-controlled account.
- Repoint the `AccessControlManager`, effectively rewriting which accounts can call all ACM-gated PrimeV2 administrative functions.
- Prevent governance from recovering normal PrimeV2 administration if ownership is not correctly transferred and accepted by the intended timelock.

PrimeLeaderboard

In the contract `PrimeLeaderboard`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call administrative functions including:

- `setPrimeV2(address)`
- `setMultiplierTiers(uint256[], uint256[])`
- `initializeStakers(address[], uint256[])`
- `finalizeInitialization()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted these privileges may allow the attacker to take advantage of this authority and do the following:

- Replace the `PrimeV2` address called by `xvsUpdated()`, causing vault stake updates to notify an attacker-controlled Prime contract or a broken integration.
- Reconfigure multiplier tiers used by `getEffectiveStake()`, changing leaderboard ranking and Prime mint eligibility.
- Configure extreme or malformed multiplier schedules that overstate or understate effective stake, distort rankings, or increase gas costs in leaderboard reads and compaction.
- Seed arbitrary initial staker balances during migration if initialization is still open.

- Finalize initialization early, preventing legitimate migration batches from being applied afterward.

In the contract `PrimeLeaderboard`, the `owner` account also has authority inherited through the upgrade/access-control base contracts, including ownership transfer and access-control-manager configuration paths.

Any compromise to the `owner` account may allow the attacker to take advantage of this authority and do the following:

- Transfer ownership to an attacker-controlled account.
- Repoint the `AccessControlManager`, changing who can configure leaderboard tiers, migration seeding, and PrimeV2 wiring.
- Retain deployer-side control if the ownership handoff is started but the intended timelock does not call `acceptOwnership()`.

Deployment and Configuration Scripts

The deployment and configuration scripts are responsible for deploying `PrimeV2` and `PrimeLeaderboard`, wiring them to each other and to external Venus components, granting the required `AccessControlManager` permissions, and handing ownership to the intended governance address.

Any compromise or mistake in the deployment operator account, deployment configuration, resolved deployment artifacts, or chain-specific admin/timelock addresses may allow the attacker to take advantage of this authority and do the following:

- Deploy or initialize Prime contracts with incorrect external dependencies.
- Leave PrimeV2, PrimeLeaderboard, or their external integrations partially configured or unwired.
- Grant sensitive permissions to an unintended account or fail to grant permissions required for normal operation.
- Start an ownership handoff without completing it through the required acceptance step.
- Route governance, ownership, or operational authority to an incorrect address.

Because deployment scripts execute with elevated operational authority and rely on chain-specific configuration, their failure modes are broad and deployment-dependent.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Venus Labs, 06/08/2026]: Issue acknowledged. I won't make any changes for the current version. ACM permissions are assigned to specific timelocks based on the required level of control, normal, fast-track, or critical, and a guardian is designated for critical setters. All permission assignments are executed through a VIP and approved via Venus governance, accompanied by a community post to ensure transparency to users. And the default admin role is exclusively owned by the normal timelock only.

VLP-02 | Residual Reward Claims Can Clear Unindexed PLP Rewards

Category	Severity	Location	Status
Logical Issue	● Medium	contracts/Tokens/Prime/PrimeV2.sol (base_old_prime_9a6aa2a): 940~941, 959~960	● Resolved

Description

`PrimeV2._claimInterest()` only calls `accrueInterest(vToken)` when the market exists and the claimant is currently a Prime holder. However, users can retain residual `interests[vToken][user].accrued` after losing Prime status, and residual accrued balances can also remain after a market is removed. In those paths, `_claimInterest()` skips `accrueInterest(vToken)` but can still call `IPrimeLiquidityProvider(primeLiquidityProvider).releaseFunds(address(asset))` if PrimeV2 does not have enough local balance to pay the claim.

This creates an accounting gap because `_claimInterest()` deletes `unreleasedPLPIncome[underlying]` immediately before calling `releaseFunds()`, while `PrimeLiquidityProvider.releaseFunds()` performs its own accrual and then clears PLP accounting:

```
accrueTokens(token_);
uint256 accruedAmount = _tokenAmountAccrued[token_];
delete _tokenAmountAccrued[token_];
IERC20Upgradeable(token_).safeTransfer(_prime, accruedAmount);
```

If time has passed since PrimeV2 last accrued the market, `releaseFunds()` can add fresh rewards to `_tokenAmountAccrued[token_]`, transfer the accrued amount to PrimeV2, and then delete the PLP accrued balance. Because `PrimeV2.accrueInterest()` was skipped, `market.rewardIndex` is not increased, and the released amount is not explicitly tracked as undistributed or non-distributable.

The primary impact path is a residual claim by a former Prime holder while the market is still active. The claimant is no longer a Prime holder, so `_claimInterest()` does not accrue the market before paying their stored `accrued` balance. If PrimeV2 has insufficient local balance, the residual claim resets `unreleasedPLPIncome[underlying]` and releases funds from PLP. Any fresh PLP rewards accrued during that release are transferred and cleared without being indexed for the market's current active Prime participants.

A secondary, more conditional case exists for removed markets. Because `removeMarket()` requires `sumOfMembersScore == 0`, this is not a direct current-holder underpayment path at the time of removal. However, a residual claim for a removed market can still reset `unreleasedPLPIncome[underlying]` and trigger PLP release, causing fresh PLP accrual for the underlying token to be transferred and cleared without a market reward-index update or explicit undistributed accounting. If the same underlying market is later reintroduced, those cleared rewards are no longer available to be distributed through the new market's `rewardIndex` or otherwise handled by PrimeV2 accounting.

As a result, PLP rewards that should either be indexed for active Prime holders or explicitly accounted as undistributed/non-distributable can become silent excess balance in PrimeV2. In the active-market case, current Prime holders can be

underpaid because PLP accrued balance is cleared without a corresponding increase to `market.rewardIndex`.

Scenario

1. Market `vTokenA` is active and has current Prime holders Bob and Carol.
2. Alice was previously a Prime holder and has `interests[vTokenA][Alice].accrued = 100`.
3. Alice is burned, so `isPrimeHolder[Alice] == false`, but her residual accrued rewards remain claimable.
4. At this point, PLP has already accrued `100` tokens that were indexed by Prime.
5. Time passes such that another `10` tokens become accrualable by PLP for the same underlying asset. These `10` have not yet been added to PLP accounting, but would be accrued the next time `PLP accrueTokens()` is called, and should belong to current Prime holders Bob and Carol.
6. Alice calls `claimInterest(vTokenA)`.
7. Because Alice is no longer a Prime holder, `_claimInterest()` skips the `accrueInterest(vTokenA)` call.
8. Prime has insufficient local balance, so it calls `releaseFunds()`.
9. PLP's `releaseFunds()` calls `accrueTokens()`, includes the new `10` tokens, transfers `110` tokens to Prime, and deletes `_tokenAmountAccrued[token]`.
10. Prime pays Alice her `100`, but the additional `10` was never added to `market.rewardIndex`.
11. Bob and Carol cannot claim that `10` as rewards because PLP's accrued balance has already been cleared.

Recommendation

We recommend ensuring that `_claimInterest()` only calls `PrimeLiquidityProvider.releaseFunds()` after the PLP amount being released has been accounted for by PrimeV2.

For existing markets, `_claimInterest()` should call `accrueInterest(vToken)` before any PLP release, regardless of whether the claimant is currently a Prime holder. This ensures fresh PLP accrual is either added to `market.rewardIndex` for active scored participants or explicitly handled under the zero-score/non-distributable accounting rule before PLP state is cleared.

For removed markets, residual claims should not pull fresh PLP accrual through the normal release path unless PrimeV2 has a defined accounting treatment for that amount. Prefer paying residual claims from PrimeV2's existing balance or from amounts already snapshotted/accounted before removal. If additional PLP funds must be released, the released amount should be explicitly tracked as undistributed, reserved, or non-distributable instead of becoming silent excess balance in PrimeV2.

Alleviation

[CertiK, 06/08/2026]: The client made the recommended changes resolving this finding in commit [3b8b10a1e2f45f0d0f6b7a144a71f2ac39ac7ef6](#).

However, we leave this finding pending to ensure the following comments are seen:

If the PLP has a nonzero distribution speed remaining for a token after its market is removed, then if residual rewards are left and claimed causing `releaseFunds()` to be called, it can flush the reward without adding it to undistributedReward. Can

you please clarify if you will ensure operationally that a market will only be removed after its underlying distribution is zeroed in the PLP and that an underlying distribution speed will be made nonzero only after a market is (re)added to ensure that while a market does not exist it cannot accrue any PLP rewards.

[Venus Labs, 06/10/2026]: We will enforce both ordering invariants operationally, since `PrimeV2.removeMarket`, `PrimeV2.addMarket`, and `PrimeLiquidityProvider.setTokensDistributionSpeed` are all governance/ACM-gated and coordinated in the same VIP / keeper runbook:

1. Remove only after zeroing distribution before `removeMarket(vToken)`, set the underlying's speed to 0 via `setTokensDistributionSpeed([underlying],[0])`; `removeMarket` then runs its `accrueInterest` flush, capturing any final slice into `undistributedReward` (sweepable).
2. Nonzero speed only after (re)add `setTokensDistributionSpeed([underlying],[nonzero])` is issued only after `addMarket(vToken,...)`. So while a market does not exist, its underlying accrues no new PLP rewards, and the `residual-claim releaseFunds()` path cannot flush a fresh unindexed slice. This ordering will be enforced in the Prime market add/remove runbook.

VLP-03 | Partial Score Updates Can Misallocate Prime Rewards

Category	Severity	Location	Status
Design Issue	● Medium	contracts/Tokens/Prime/PrimeV2.sol (base_prime_e421dda): 494 ~497, 576~610	● Acknowledged

Description

The `PrimeV2.updateScores` function is used to refresh Prime holder scores after governance changes a parameter that affects score calculation, such as `updateAlpha`, `updateMultipliers`, or `addMarket`.

When such a change happens, the contract does not update all Prime holders immediately. Instead, it opens a pending score-update round by setting `pendingScoreUpdates` and waits for users to be updated later through calls to `updateScores(address[] users)`.

During this migration window, the contract can enter a temporary mixed state where some users have already been recalculated under the new parameters while other users still retain stale scores from the old configuration. Rewards continue to accrue and be claimed during this period. Since `PrimeV2.accrueInterest` distributes newly accrued rewards based on `market.sumOfMembersScore`, which is the sum of currently stored user scores, rewards can be split using an inconsistent combination of old and new scores rather than a single coherent snapshot.

This mixed state can also be created or accelerated through the public `accrueInterestAndUpdateScore(address user, address market)` function. Although this function does not change protocol parameters itself, after governance has already changed the parameters it allows any caller to selectively recalculate chosen users under the new scoring rules before the queued score-update round is complete. While the `accrueInterestAndUpdateScore(address user)` overload is now restricted to `PrimeLeaderboard`, the market-specific entrypoint remains callable through normal market hooks and can still move selected users onto the new scoring state ahead of the rest of the holder set.

As a result, rewards accrued during this partial-update window can be split incorrectly. A user whose score increases can be updated early and receive a larger share of rewards while other users remain stale. Conversely, a user whose score decreases can remain over-weighted until someone updates them, or can be forced onto the lower score early while others remain on old scores.

The `addMarket` path is the clearest example because the new market becomes active before all existing Prime holders are initialized into it, so early-updated holders can receive rewards while later-updated holders still have zero score in that market.

Functions such as `updateMultipliers` settle rewards before applying the new configuration, so rewards accrued before the change use the old scores. Rewards accrued after the change can still be misallocated because users are migrated gradually and the denominator can contain a mix of old and new scores.

Scenario

1. Alice and Bob are both Prime holders with equal stored scores.

2. Governance calls `updateMultipliers` or `updateAlpha`, changing the score formula.
3. The contract opens a score-update round by setting `pendingScoreUpdates`.
4. Under the new formula, Alice's score should increase while Bob's score should stay the same or increase less.
5. Alice, or any third party, calls `updateScores([Alice])` or `accrueInterestAndUpdateScore(Alice, market)`.
6. Alice is updated to her new higher score, while Bob remains stale with his old score.
7. New PLP rewards accrue before Bob is updated.
8. Alice calls `claimInterest`, or anyone calls `accrueInterest`.
9. The newly accrued rewards are distributed using Alice's new score and Bob's old score.
10. Alice receives a larger reward share than she would have received under a consistent old snapshot or a fully migrated new snapshot.
11. Bob is underpaid for that accrual interval until someone later updates him, and the earlier reward split is not clawed back.

Recommendation

We recommend ensuring that reward accounting cannot use a partially migrated score set. For example, score-affecting configuration changes could be applied through an epoch or snapshot mechanism where rewards accrued before the score-update round is complete continue to use the previous score epoch, and rewards accrued after completion use the new score epoch.

Alternatively, reward accrual and claiming could be paused or deferred while `pendingScoreUpdates > 0`, so no rewards are distributed using a mix of old and new scores.

Any function that recalculates user scores during an active score-update round should also participate in the same round accounting. In particular, if `accrueInterestAndUpdateScore(address user, address market)` updates a user under the new parameters while `pendingScoreUpdates > 0`, the contract should either mark that user as updated for the current round or defer applying the new score until the user is processed by `updateScores()`. This avoids selectively moving chosen users onto the new scoring rules ahead of the rest of the holder set.

Alleviation

[Venus Labs, 06/08/2026]: Issue acknowledged. I won't make any changes for the current version. This is a temporary, zero-sum reward redistribution confined to the score-update window; no funds are lost, stolen, or stranded, and the total reward pool is preserved. Parameter changes that open such a window (`updateAlpha`, `updateMultipliers`, `addMarket`) are infrequent and executed deliberately, and the keeper finalizes the `updateScores` round in the same block as the change. As a result, the window is effectively nonexistent, and the reward accrued against a mixed `sumOfMembersScore` is too small to matter. Additionally, the holder set cannot change mid-round, since `issue/burn/claimPrime` and their batch variants revert with `ScoreUpdateInProgress`. Fully eliminating the residual would require a score-epoch redesign of the reward-index accounting, which we consider disproportionate to the negligible impact.

VLP-04 Tier-Based Deposit Compaction Can Understate Remaining Effective Stake After Partial Withdrawals

Category	Severity	Location	Status
Design Issue	Minor	contracts/Tokens/Prime/PrimeLeaderboard.sol (base_prime_e421dda): 369~372, 393~407, 540~548	Acknowledged

Description

The `PrimeLeaderboard` contract compacts deposits by merging multiple deposits in the same current multiplier tier into a single synthetic `Deposit` with an amount-weighted average timestamp. Later, withdrawals are still processed in LIFO order against the stored deposit stack.

This creates an accounting mismatch. After compaction, the contract no longer preserves the original tranche ordering or timestamps of the merged deposits. If a later withdrawal removes only part of a merged entry, the remaining balance keeps the averaged timestamp of the synthetic deposit, even though true LIFO accounting would have removed the newest underlying portion first and left an older residual balance behind.

As a result, the remaining position can be recorded as younger than it should be, which causes `getEffectiveStake` to understate the user's effective stake after compaction and a later partial withdrawal. If this averaged-residual behavior is intentional, it should be explicitly documented as a consequence of compaction, because it does not preserve true tranche-level LIFO aging semantics.

Scenario

We assume:

1. A user makes three deposits that later fall into the same multiplier tier:
 - 100 at day 0
 - 100 at day 10
 - 100 at day 20
2. The user reaches `MAX_DEPOSITS_PER_USER`, so `_recordDeposit` triggers compaction and `_compactByTier` merges those deposits into one synthetic entry:
 - 300 with an averaged timestamp near day 10
3. Later, the user withdraws 100, and `_recordWithdrawal` applies LIFO against the compacted entry.
4. The contract leaves:
 - 200 with the same averaged timestamp near day 10

5. Under true tranche-level LIFO accounting, the newest `100` from day `20` should have been removed first, leaving:
- `100` from day `0`
 - `100` from day `10`
6. The true remaining position is therefore older than the compacted `200 from day 10` representation stored by the contract.
7. When `getEffectiveStake(user)` is called, the remaining balance is treated as younger than it should be, so the user's effective stake is understated.

Recommendation

We recommend either preserving tranche-age information during compaction so that later partial withdrawals produce the same residual age profile as true tranche-level LIFO accounting, or explicitly documenting that compaction replaces tranche-accurate LIFO aging with averaged residual timestamps. If tranche-accurate effective stake is required, timestamp-averaged compaction should not be used for deposits that may later be partially withdrawn.

Alleviation

[Venus Labs, 06/08/2026]: Issue acknowledged. I won't make any changes for the current version. The timestamp averaging during compaction is an intentional trade-off for bounded per-user deposit storage (capped at 30 entries via `MAX_DEPOSITS_PER_USER`). The resulting residual-age inaccuracy after a partial withdrawal is conservative and tightly bounded. The merged timestamp is ceiled, so duration always rounds down `getEffectiveStake` can only understate, never overstate, meaning a user can never gain undeserved leaderboard rank or `claimPrime` eligibility from it. Pass 1 (max-tier merge) is lossless for effective stake, since `maxTierDuration == maxCapSeconds`: every merged deposit is already at max multiplier and duration-capped, so its averaged timestamp has no effect. The inaccuracy is confined to the Pass 2 same-tier fallback followed by a partial withdrawal, where it shifts only the uncapped duration term within a single multiplier tier, and only triggers once a user reaches the 30-deposit cap. A fix preserving true per-tranche LIFO aging would require unbounded deposit history, defeating the storage cap that compaction exists to enforce. Accepted as documented, always user-conservative.

VLP-05 | Precision Truncation In `accrueInterest` Can Permanently Drop Small Reward Slices

Category	Severity	Location	Status
Coding Issue	Minor	contracts/Tokens/Prime/PrimeV2.sol (base_prime_e421dda): 466~486	Resolved

Description

In `PrimeV2.accrueInterest` function, newly accrued PLP income is converted into a market reward-index increase using integer division:

```
(distributionIncome * EXP_SCALE) / market.sumOfMembersScore
```

If the newly accrued `distributionIncome` is too small relative to `market.sumOfMembersScore`, this calculation rounds down to `0`. However, the function still advances:

```
unreleasedPLPIncome[underlying] = totalAccruedInPLP;
```

as if that reward slice had been fully accounted for.

This creates a precision-loss accounting bug, since small reward slices can be treated as already distributed even though `market.rewardIndex` did not increase. Because user rewards are derived from `rewardIndex`, those rounded-away slices are not carried into future distribution and can be permanently lost from Prime reward accounting.

Scenario

We assume:

1. A Prime market has a large `market.sumOfMembersScore`, for example `1e25`.
2. A small new PLP reward slice accrues for the market, for example `distributionIncome = 9e6` in the underlying token's smallest units.
3. A caller invokes `accrueInterest(vToken)`.
4. The contract computes $(9e6 * 1e18) / 1e25 = 0$, so `market.rewardIndex` does not increase.
5. The function still sets `unreleasedPLPIncome[underlying] = totalAccruedInPLP`, marking that reward slice as already processed.
6. On the next accrual, that slice is no longer part of `totalAccruedInPLP - unreleasedPLPIncome[underlying]`, so it is not retried or rolled forward.
7. Prime holders therefore never receive that reward slice through index-based distribution.

Recommendation

We recommend only advancing `unreleasedPLPIncome[underlying]` by the portion of rewards that is actually reflected in `market.rewardIndex`, or carrying sub-threshold residual amounts forward until enough income accumulates to produce a nonzero index increment.

■ Alleviation

[Venus Labs, 06/08/2026]: `accrueInterest` no longer consumes sub-threshold income: when the index delta truncates to 0 it returns without advancing `unreleasedPLPIncome`, so the slice is retried later; when it advances, the anchor moves only by the income actually indexed. Remainder stays pending instead of being stranded.

[CertiK, 06/08/2026]: The client made the recommended changes resolving this finding in commit [a1fc82948cc768b8e9a980b553f8b3aef6dd77a7](#), however, we leave this finding pending to ensure the following comment is seen.

The normal precision-truncation path is fixed because `accrueInterest` no longer advances `unreleasedPLPIncome` for unindexed income and remainders are carried forward until they can move `rewardIndex`.

However, a residual edge case remains when `_claimInterest()` or `sweepUndistributed()` enters a shortfall branch and calls `releaseFunds()` after deleting `unreleasedPLPIncome`. Since `releaseFunds()` clears the full PLP accrued balance for the underlying, it can also pull pending precision remainders that were intentionally left in the PLP, before those remainders are indexed into `rewardIndex` or moved to `undistributedReward`. We recommend either clarifying this is an accepted amount of unaccounted dust accrual or refactoring the logic to account for this amount during `releaseFunds()`.

[Venus Labs, 06/10/2026]: We accept this as bounded dust. The pulled remainder is by construction less than `sumOfMembersScore / EXP_SCALE` (a sub-one-unit index amount that couldn't move `rewardIndex`), occurs only when a `releaseFunds()` shortfall coincides with a pending remainder, and is conservative (under-distribution only, no theft). The pulled amount stays as PrimeV2 surplus backing later claims, so it isn't extractable. Accounting it in `releaseFunds()` would re-handle an amount that by definition can't be meaningfully indexed not worth the added complexity.

VLP-06 | Non-BSC PrimeV2 Deployments Can Fail Due To Required Unitroller Dependency Resolving To Zero Address

Category	Severity	Location	Status
Inconsistency	Minor	deploy/014-deploy-prime-v2.ts (base_prime_e421dda): 64	Acknowledged

Description

`014-deploy-prime-v2.ts` appears to support multiple networks through network-specific config entries for Ethereum, Arbitrum, Optimism, Base, Unichain, zkSync, and others. However, the script fetches the core pool comptroller using a BSC-specific deployment name:

```
const corePoolAddress = await getContractAddressOrNullAddress(deployments, "Unitroller");
```

`getContractAddressOrNullAddress` returns the zero address when the deployment artifact is missing. In this repository, `Unitroller.json` is only present for `bscmainnet` and `bsctestnet`.

The returned `corePoolAddress` is passed into `PrimeV2.initialize`:

```
corePoolAddress,
```

But `PrimeV2.initialize` rejects a zero address:

```
if (corePoolComptroller_ == address(0)) revert InvalidAddress();
```

As a result, deployments to non-BSC networks may revert during PrimeV2 initialization if no external `Unitroller` deployment is available.

Recommendation

We recommend resolving the correct comptroller or pool registry mechanism per network. Also we recommend replacing the optional lookup with a required one for mandatory dependencies.

Alleviation

[Venus Labs, 06/08/2026]: Issue acknowledged. I won't make any changes for the current version. PrimeV2 is currently BSC-only its only intended targets are bscmainnet/bsctestnet, where the Unitroller lookup resolves correctly. On non-BSC nets the zero address makes initialize revert with InvalidAddress, a safe fail-fast. If Venus deploys Prime on other chains in the future, the script will be updated then to resolve the comptroller per network as a required dependency.

VLP-07 PrimeV2 Deploy Script May Attempt ACM-Gated Wiring Before VIP Permissions Are Granted

Category	Severity	Location	Status
Inconsistency	Minor	deploy/014-deploy-prime-v2.ts (base_prime_e421dda): 165~168	Resolved

Description

The PrimeV2 deployment flow deploys `PrimeV2` and `PrimeLeaderboard`, then attempts to wire the two contracts together by calling:

```
await primeLeaderboard.setPrimeV2(primeV2.address);
await primeV2.setPrimeLeaderboard(primeLeaderboard.address);
```

Both functions are ACM-gated:

```
function setPrimeV2(address primeV2_) external {
    _checkAccessAllowed("setPrimeV2(address)");
    ...
}

function setPrimeLeaderboard(address primeLeaderboard_) external {
    _checkAccessAllowed("setPrimeLeaderboard(address)");
    ...
}
```

However, the deploy script only grants these permissions to the deployer in the `hardhat` environment. On live networks, the testnet `VIP` grants the exact same permissions to the `NORMAL_TIMELOCK` and then has the timelock perform the wiring. This means the VIP assumes these calls are governance actions, not deployer actions.

If the live deploy script still attempts to call these functions before the VIP executes, the calls will revert unless the deployer already has those ACM permissions. This can cause the deployment script to fail, or create confusion about whether PrimeV2 and PrimeLeaderboard have been wired successfully.

Recommendation

We recommend restricting the direct wiring calls in the deploy script to local testing environments, or remove them from the live deploy flow entirely. On live networks, leave `PrimeV2.setPrimeLeaderboard()` and `PrimeLeaderboard.setPrimeV2()` to the VIP, where the required ACM permissions are granted to the timelock immediately before execution.

Alleviation

[Venus Labs, 06/08/2026]: The ACM-gated wiring (setPrimeV2 / setPrimeLeaderboard) and its permission grants are now restricted to the local hardhat environment, with a comment noting that on live and testnet networks the wiring is performed by the timelock via a VIP so the deployer no longer issues calls that would revert.

Fixed in [PR #677](#).

VLP-08 | Zero Oracle Price Allows Uncapped PrimeV2 Score Capital

Category	Severity	Location	Status
Design Issue	● Informational	contracts/Tokens/Prime/PrimeV2.sol (base_prime_e421dda): 1141~1171	● Acknowledged

Description

The `PrimeV2` contract calculates a user's market score using their XVS backing and their Venus supply/borrow activity. The supply and borrow amounts are intended to be capped by the user's XVS value, limiting how much market activity can count toward Prime rewards.

The cap is only applied when `oracle.getUnderlyingPrice(market)` returns a price greater than zero. If the oracle returns zero, the function keeps the user's full supply and borrow balances as score capital. As a result, a Prime holder with positive XVS stake and a large market position can receive a score based on uncapped market balances during a zero-price oracle condition.

The inflated score would be stored in `interests[market][user].score` and included in `markets[market].sumOfMembersScore`. Future reward accrual for that market can then distribute a larger share of rewards to the incorrectly scored user, reducing the share received by other Prime holders.

Venus' `ResilientOracle` treats zero prices as invalid and reverts with `"invalid resilient oracle price"` when no valid non-zero price can be returned. This means the described zero-price path is not expected to occur with the standard `ResilientOracle` configuration. Still, explicitly rejecting zero prices in `PrimeV2` would make the score-cap invariant local to the contract and avoid relying on every compatible oracle implementation to enforce the same behavior.

Recommendation

We recommend defensively treating a zero market price as invalid during PrimeV2 score calculation. The function should either revert when `oracle.getUnderlyingPrice(market)` returns zero or set the user's supply and borrow capital to zero for that market until a valid price is available. This is a best-practice hardening measure to make the score-cap behavior explicit in `PrimeV2`.

Alleviation

[Venus Labs, 06/08/2026]: Issue acknowledged. I won't make any changes for the current version. The zero-price branch is unreachable with Venus' standard `ResilientOracle`. `_calculateScore` refreshes the price via `oracle.updatePrice(market)` and reads it through `getUnderlyingPrice`, which delegates to `_getPrice`. That function treats 0 as `INVALID_PRICE` and returns only a validated, strictly positive price; if no valid non-zero price is available from the main/pivot/fallback oracles it reverts with `"invalid resilient oracle price"` rather than returning 0. Consequently `getUnderlyingPrice` can never return 0 to `_capitalForScore`, so the uncapped-capital path described cannot occur under the configured oracle.

VLP-09 Required Timelock Address Resolution Fails Open To Zero Address

Category	Severity	Location	Status
Inconsistency	● Informational	deploy/014-deploy-prime-v2.ts (base_prime_e421dda): 85~86; deploy/015-configure-prime-v2.ts (base_prime_e421dda): 27~28	● Acknowledged

Description

In `deploy/014-deploy-prime-v2.ts` script, most configured networks assign the proxy admin owner through explicit hardcoded governance addresses. By contrast, `bscmainnet` and `bsctestnet` derive the owner from `getContractAddressOrNullAddress(deployments, "NormalTimelock")`.

This creates a risky failure mode because `getContractAddressOrNullAddress` is a nullable helper: when the requested deployment record is unavailable, it catches the error and returns `0x0000000000000000000000000000000000000000` instead of reverting. That behavior is appropriate for optional addresses, but `NormalTimelock` is a required privileged address.

In `deploy/014-deploy-prime-v2.ts`, the resolved BSC `NormalTimelock` value is used as the proxy admin owner for both `PrimeLeaderboard` and `PrimeV2`. If the deployment record is missing or incorrectly configured, the deployment can continue with `address(0)` as the proxy admin owner instead of failing immediately.

The same issue appears in `deploy/015-configure-prime-v2.ts`, where `bscmainnet` and `bsctestnet` again resolve `NormalTimelock` through `getContractAddressOrNullAddress`. The resolved value is then passed to `primeV2.transferOwnership()` and `primeLeaderboard.transferOwnership()`. If the lookup falls back to `address(0)`, the ownership handoff does not revert in these contracts. It sets `pendingOwner` to `address(0)`, leaving the current owner unchanged and making the intended timelock ownership transfer impossible to finalize.

This makes the BSC deployment path dependent on external deployment metadata for a required governance address while failing open when that metadata is missing. In the intended deployment environment this lookup may resolve correctly, but the script does not enforce that requirement before using the result in privileged ownership configuration.

Recommendation

We recommend avoiding `getContractAddressOrNullAddress` for required privileged deployment inputs. The BSC deployment path should resolve `NormalTimelock` through a fail-closed mechanism and explicitly validate that the resolved address is not `address(0)` before using it as a proxy admin owner or ownership-transfer target. The `getContractAddressOrNullAddress` helper should be reserved for optional dependencies only, while missing governance/admin addresses should cause deployment to revert immediately.

Alleviation

[Venus Labs, 06/08/2026]: `NormalTimelock` is a long-standing core governance contract on both `bscmainnet` and `bsctestnet`, so `getContractAddressOrNullAddress` always resolves it to the correct non-zero address during the intended

Prime deployments; the zero-address fallback is never hit in practice. The non-BSC entries use hardcoded governance addresses and are unaffected. As Prime is currently BSC-only and the timelock deployment is guaranteed to exist on the target networks, we accept the finding. Should this ever change, the deployment scripts will resolve the timelock as a required, non-nullable dependency.

VLP-10 PrimeV2 Deploy Script Uses A Local Deployment Dependency In Live Deployments

Category	Severity	Location	Status
Logical Issue	● Informational	deploy/014-deploy-prime-v2.ts (base_prime_e421dda): 176	● Resolved

Description

`014-deploy-prime-v2.ts` declares the legacy Prime deployment tag as a dependency:

```
func.dependencies = ["Prime"];
```

This dependency is useful for fresh local `hardhat` deployments because it ensures that the legacy `Prime` deployment script runs first and creates the `PrimeLiquidityProvider` needed by PrimeV2.

However, on live networks, `PrimeLiquidityProvider` is expected to already exist and be controlled by governance. The PrimeV2 script already fetches it directly:

```
const plp = await ethers.getContract("PrimeLiquidityProvider");
```

which will fail clearly if the deployment artifact is missing.

Therefore, keeping the full `Prime` tag dependency in live deployments unnecessarily reruns the legacy Prime deployment flow. Since that flow manages the legacy `Prime` and `PrimeLiquidityProvider` proxy deployments, it can cause avoidable deployment failures or proxy-admin interactions after ownership has already moved to the Timelock.

Recommendation

We recommend removing the broad `Prime` tag dependency from the live PrimeV2 deployment flow. PrimeV2 only requires the existing `PrimeLiquidityProvider` address, so the script should fetch that deployment artifact directly and fail with a clear error if it is unavailable or does not match the expected network configuration.

If a dependency on the legacy Prime deployment flow is useful for local or fresh test deployments, keep it behind an explicit local/fresh-deployment path rather than applying it to live networks. This avoids unnecessary coupling to legacy Prime deployment logic while preserving developer convenience in clean environments.

Alleviation

[Venus Labs, 06/08/2026]: The legacy ["Prime"] deployment dependency has been removed from the PrimeV2 deploy flow in [PR #677](#).

PrimeV2 now relies on the existing `PrimeLiquidityProvider` fetched directly in the script, which fails clearly if the artifact is unavailable, rather than rerunning the legacy Prime deployment on live networks.

VLP-11 PrimeLeaderboard Can Be Initialized With Stale Or Inconsistent State During VIP Migration

Category	Severity	Location	Status
Logical Issue	● Informational	contracts/Tokens/Prime/PrimeLeaderboard.sol (base_prime_e42 1dda): 103	● Resolved

Description

`PrimeLeaderboard` includes a migration flow for seeding existing XVSVault stakers:

```
initializeStakers(users, amounts, timestamps)
finalizeInitialization()
```

The contract comments describe the intended process as:

1. Deploy PrimeLeaderboard
2. Call initializeStakers() in batches to seed existing staker data
3. Call finalizeInitialization() to lock seeding permanently
4. Point XVSVault.primeToken to this contract to begin live tracking

However, the testnet VIP configures the live `xvsvault` hook before staker seeding is completed. Specifically, the VIP calls:

```
XVSVault.setPrimeToken(PrimeLeaderboard, XVS, XVS_VAULT_POOL_ID)
```

but states that staker seeding is not performed in the VIP and will instead be performed later by the Guardian using off-chain snapshot data.

`initializeStakers()` trusts the supplied `amounts` and `timestamps`, and does not validate them against the current XVSVault balance at execution time. In addition, the function skips users that already have a nonzero `totalStaked` value:

```
if (totalStaked[user] == 0 && amounts[i] > 0) {
    ...
}
```

If the VIP ordering is followed, where `xvsvault` is pointed to `PrimeLeaderboard` before Guardian seeding is complete:

- If an existing unseeded staker partially withdraws, `xvsUpdated()` records the remaining stake as a fresh deposit, losing the historical staking timestamp. If the user later calls `claimPrime()`, their eligibility is calculated from the newer timestamp and may be understated.
- If an existing unseeded staker deposits more, `xvsUpdated()` may record the full current balance as a fresh deposit, losing the historical age of the existing stake. If the user later calls `claimPrime()`, their score may not

reflect the true age of their original stake.

- If an existing unseeded staker fully withdraws, `xvsUpdated()` records no state because both the current vault stake and `totalStaked[user]` are zero. A stale Guardian snapshot could later seed that user with stake they no longer hold, allowing `claimPrime()` to evaluate eligibility using stale stake data.
- If a user is live-updated before Guardian seeding, later `initializeStakers()` will skip them because `totalStaked[user] != 0`, preventing the historical snapshot timestamp from being applied. If the user later calls `claimPrime()`, their eligibility may be based on the live update timestamp rather than their historical staking duration.

If the contract-comment ordering is followed instead, where stakers are seeded before `XVSVault` is pointed to `PrimeLeaderboard`, there is still a consistency risk during the period between snapshot generation and hook activation:

- If a user is snapshotted with 100 XVS and then fully withdraws before `XVSVault` is switched to `PrimeLeaderboard`, the withdrawal is sent to the legacy Prime hook, not `PrimeLeaderboard`. The user may still be seeded with 100 XVS, allowing `claimPrime()` to evaluate eligibility using stake they no longer hold.
- If a user is snapshotted with 100 XVS and then partially withdraws to 50 XVS before hook activation, `PrimeLeaderboard` may be initialized with the stale 100 XVS amount until the user later triggers `xvsUpdated()`. Until then, `claimPrime()` may use an inflated score.
- If a user stakes additional XVS after the snapshot but before hook activation, the additional stake is not reflected in the seeded data, causing the user to be underrepresented. If the user calls `claimPrime()` before a later vault update, eligibility may be calculated from an understated score.
- If users can predict the snapshot or Guardian seeding window, they may frontrun the process by temporarily increasing stake before the snapshot or withdrawing after the snapshot but before the initialized values are finalized. Since `claimPrime()` reads `PrimeLeaderboard.getEffectiveStake(user)` and does not refresh from `XVSVault`, stale initialized values can directly affect claim eligibility.

In either ordering, user interactions with `XVSVault` during the migration window can cause `PrimeLeaderboard` to store stale or incorrect stake amounts and timestamps, directly affecting `claimPrime()` eligibility.

One potential mitigation is to pause `XVSVault` during the migration window. Since the Guardian is intended to perform staker initialization, it may also be intended to coordinate this pause/unpause process.

Can the team clarify the exact migration sequence that will be followed, including whether `XVSVault` will be paused during snapshot generation, `initializeStakers()`, `finalizeInitialization()`, and the switch to `PrimeLeaderboard`?

Recommendation

We recommend clarifying the above.

Alleviation

[Venus Labs, 06/08/2026]: Migration runs as one atomic VIP, then Guardian seeding under a paused vault: VIP (atomic):

(1) deploy PrimeV2 + PrimeLeaderboard.

(2) wire them together

(3) point XVSVault.primeToken to the new contract

(4) pause XVSVault deposits/withdrawals.

After VIP:

(5) Guardian calls initializeStakers() in batches then finalizeInitialization()

(6) a multisig tx / VIP unpauses XVSVault.

Steps 3 and 4 are atomic in the same VIP, so no stake change can interleave between the repoint and the pause; with the vault paused throughout seeding, xvsUpdated cannot fire and seeding runs against a frozen state. Seeding is also idempotent (totalStaked != 0 skipped) and timestamp-validated.

APPENDIX | VENUS LABS - PRIMEV2 AND PRIME LEADERBOARD CONTRACTS

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Venus Labs - PrimeV2 and Prime Leaderboard contracts Security
Assessment

| CertiK Assessed on Jun 10th, 2026 | Copyright © CertiK

